INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DO CEARÁ
CAMPUS BOA VIAGEM
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS

Fernanda Lara Alves de Oliveira

Feira Digital do Sertão

Sistema web orientado a objetos em PHP

Relatório do trabalho individual da disciplina Programação Web I
Professor: Prof. Dr. Renato William Rodrigues de Souza

Boa Viagem, CE
2026

Sumário

1 Introdução **2**

2 Requisitos **2**

2.1 Funcionalidades 2

2.2 Regras de negócio 3

3 Modelagem **3**

3.1 Herança 4

3.2 Composição 4

4 Implementação **4**

4.1 Encapsulamento e construtores (OO1 e OO4) 4

4.2 Herança e sobrescrita (OO2) 5

4.3 Polimorfismo (OO3) 6

4.4 Regras de negócio com exceções (OO5) 7

4.5 Composição (OO6) 7

4.6 Organização em arquivos e conversão para JSON (OO7 e OO8) 8

5 Camada web **8**

6 Testes **9**

7 Ferramentas utilizadas **10**

8 Uso de LLM **10**

9 Dificuldades, aprendizados e conclusão **11**

1 Introdução

A Feira Digital do Sertão é um pequeno sistema web pensado para feirantes e compradores do sertão cearense que hoje negociam seus produtos (queijos, rapaduras, carnes de sol, méis, manteigas e doces) apenas de forma presencial. O problema que o sistema resolve é a falta de um catálogo digital simples, no qual o comprador consiga montar um pedido escolhendo produtos unitários, produtos vendidos por peso e produtos perecíveis com validade curta, vendo o valor total calculado de forma automática e coerente com a natureza de cada item.

O sistema serve a dois públicos: o **feirante**, que precisa cadastrar produtos com preços, unidades e prazos de validade, e o **comprador**, que precisa montar um pedido e conferir o total antes de finalizar. Toda a lógica é orientada a objetos em PHP, com hierarquia de produtos, composição entre pedido e itens e regras de negócio explícitas que recusam dados inválidos.

Este relatório está organizado em nove seções. A Seção 2 apresenta as funcionalidades e as regras de negócio; a Seção 3 traz o diagrama de classes e justifica as heranças e a composição; a Seção 4 mostra onde cada requisito OO1–OO8 aparece no código; a Seção 5 descreve a camada web (rotas, 404, formulário e JSON); a Seção 6 apresenta as saídas do `teste.php`; a Seção 7 lista as ferramentas; a Seção 8 resume o uso de LLM; a Seção 9 discute dificuldades, aprendizados e conclusão.

2 Requisitos

2.1 Funcionalidades

- Página inicial (`?pagina=home`) com apresentação da feira.
- Página de catálogo (`?pagina=produtos`) listando os produtos cadastrados com seus tipos e preços.
- Página de pedido (`?pagina=pedido`) com formulário `POST` para adicionar um produto ao pedido atual.
- Rota `?formato=json` que devolve todos os produtos e pedidos serializados com `json_encode`.
- Páginas inexistentes (`?pagina=inexistente`) devolvem **HTTP 404** com uma mensagem amigável.
- Persistência bônus: os pedidos criados pelo formulário continuam disponíveis na próxima requisição, porque são salvos em `dados.json` com `file_put_contents` e relidos com `json_decode`.

2.2 Regras de negócio

Nº	Regra	Onde está (classe e método)
1	Preço do produto deve ser maior que zero	`Produto::setPreco()`
2	Quantidade do item deve ser maior que zero	`ItemPedido::__construct()`
3	Produto perecível vencido não pode entrar no pedido	`Pedido::adicionarItem()`
4	Pedido não pode ser finalizado vazio	`Pedido::finalizar()`
5	Peso (kg) deve ser maior que zero e no máximo 50	`ProdutoPorPeso::__construct()`

3 Modelagem

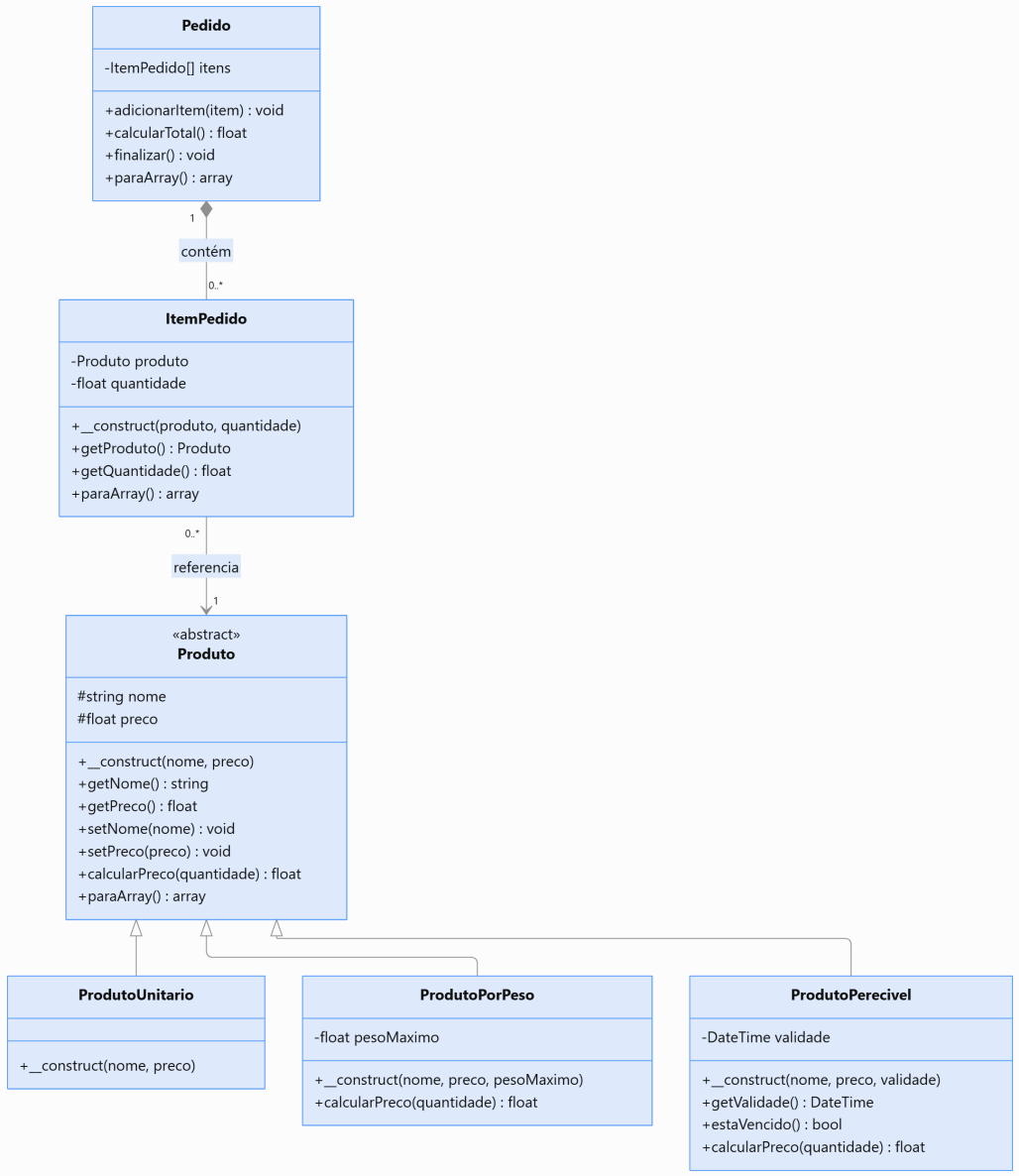


Figura 1: Diagrama de classes do sistema

3.1 Herança

A classe `Produto` é a mãe e concentra os atributos comuns a qualquer item vendido na feira: `nome` e `preco`, ambos `protected`, além do método `calcularPreco(float $qtd)`. Dela herdam três classes:

- `ProdutoUnitario` — "é um" produto vendido por unidade (queijo, rapadura). Não acrescenta atributos; apenas herda o cálculo padrão.
- `ProdutoPorPeso` — "é um" produto vendido por quilo (carne de sol, mel). Acrescenta `pesoMaximo` e sobrescreve `calcularPreco()` para arredondar o valor em duas casas e validar o peso.
- `ProdutoPerecivel` — "é um" produto com validade curta (manteiga, doce). Acrescenta `validade` (um `DateTime`) e sobrescreve `calcularPreco()` para aplicar 20% de desconto quando faltam três dias ou menos para o vencimento.

O método sobrescrito é, portanto, `calcularPreco()`, presente em `Produto` e redefinido em `ProdutoPorPeso` e `ProdutoPerecivel`. Todas as filhas chamam `parent::__construct()` para reaproveitar a inicialização de `nome` e `preco` feita pela mãe.

3.2 Composição

`Pedido` "tem um" conjunto de `ItemPedido` — a relação é de composição, e não de herança, porque um pedido não *é* um item, mas *contém* vários itens. Cada `ItemPedido` aponta para um `Produto` e guarda a quantidade desejada. O `Pedido` expõe `adicionarItem(ItemPedido $item)` e `calcularTotal()`, que percorre a lista interna chamando `calcularPreco()` de cada produto — é exatamente aí que o polimorfismo aparece em ação.

4 Implementação

4.1 Encapsulamento e construtores (OO1 e OO4)

Todos os atributos são `private` ou `protected`; não há nenhum atributo `public`. O acesso se dá por getters, e os setters existem apenas onde há validação (por exemplo, `setPreco()`).

```
<?php
declare(strict_types=1);

class Produto
{
    protected string $nome;
    protected float  $preco;
```

```
    public function __construct(string $nome, float $preco)
    {
        $this->setNome($nome);
        $this->setPreco($preco);
    }

    public function getNome(): string { return $this->nome; }
    public function getPreco(): float { return $this->preco; }

    public function setNome(string $nome): void
    {
        if (trim($nome) === '') {
            throw new Exception("Nome do produto não pode ser vazio.");
        }
        $this->nome = $nome;
    }

    public function setPreco(float $preco): void
    {
        if ($preco <= 0) {
            throw new Exception("Preço deve ser maior que zero.");
        }
        $this->preco = $preco;
    }

    public function calcularPreco(float $quantidade): float
    {
        return $this->preco * $quantidade;
    }

    public function paraArray(): array
    {
        return [
            'tipo'  => static::class,
            'nome'  => $this->nome,
            'preco' => $this->preco,
        ];
    }
}
```

4.2 Herança e sobrescrita (OO2)

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/Produto.php';
```

```
class ProdutoPerecivel extends Produto
{
    private DateTime $validade;

    public function __construct(string $nome, float $preco, DateTime $validade)
    {
        parent::__construct($nome, $preco);
        $this->validade = $validade;
    }

    public function getValidade(): DateTime { return $this->validade; }

    public function estaVencido(): bool
    {
        return $this->validade < new DateTime('today');
    }

    public function calcularPreco(float $quantidade): float
    {
        $base = parent::calcularPreco($quantidade);
        $dias = (new DateTime('today'))->diff($this->validade)->days;
        return $dias <= 3 ? round($base * 0.8, 2) : $base;
    }

    public function paraArray(): array
    {
        return parent::paraArray() + [
            'validade' => $this->validade->format('Y-m-d'),
        ];
    }
}
```

4.3 Polimorfismo (OO3)

O `foreach` abaixo percorre objetos de classes diferentes (`ProdutoUnitario`, `ProdutoPorPeso`, `ProdutoPerecivel`) e chama o mesmo método `calcularPreco()`; cada um responde do seu jeito.

```
$produtos = [
    new ProdutoUnitario('Queijo coalho', 35.00),
    new ProdutoPorPeso('Carne de sol', 42.90),
    new ProdutoPerecivel('Manteiga da terra', 28.00,
                         new DateTime('+2 days')),
];
```

```
foreach ($produtos as $p) {
    printf("%-22s R$ %8.2f\n", $p->getNome(), $p->calcularPreco(2));
}
```

4.4 Regras de negócio com exceções (OO5)

```
// Em ItemPedido::__construct
if ($quantidade <= 0) {
    throw new Exception("Quantidade deve ser maior que zero.");
}

// Em Pedido::adicionarItem
if ($item->getProduto() instanceof ProdutoPerecivel
    && $item->getProduto()->estaVencido()) {
    throw new Exception("Produto perecível vencido não pode ser vendido.");
}

// Em Pedido::finalizar
if (empty($this->itens)) {
    throw new Exception("Pedido não pode ser finalizado vazio.");
}
```

E o `try/catch` que as trata no `index.php`:

```
try {
    $pedido->adicionarItem(new ItemPedido($produto, $qtd));
    $mensagem = "Item adicionado com sucesso.";
} catch (Exception $e) {
    $erro = $e->getMessage();
}
```

4.5 Composição (OO6)

```
class Pedido
{
    private array $itens = [];

    public function adicionarItem(ItemPedido $item): void
    {
        if ($item->getProduto() instanceof ProdutoPerecivel
            && $item->getProduto()->estaVencido()) {
            throw new Exception("Produto perecível vencido.");
        }
        $this->itens[] = $item;
    }
```

```
    public function calcularTotal(): float
    {
        $total = 0.0;
        foreach ($this->itens as $item) {
            $total += $item->getProduto()
                           ->calcularPreco($item->getQuantidade());
        }
        return round($total, 2);
    }

    public function paraArray(): array
    {
        return [
            'itens' => array_map(
                fn(ItemPedido $i) => $i->paraArray(),
                $this->itens
            ),
            'total' => $this->calcularTotal(),
        ];
    }
}
```

4.6 Organização em arquivos e conversão para JSON (OO7 e OO8)

Cada classe ocupa um arquivo próprio na pasta `src/`, e o `index.php` carrega todas com `require_once`:

```
require_once __DIR__ . '/../src/Produto.php';
require_once __DIR__ . '/../src/ProdutoUnitario.php';
require_once __DIR__ . '/../src/ProdutoPorPeso.php';
require_once __DIR__ . '/../src/ProdutoPerecivel.php';
require_once __DIR__ . '/../src/ItemPedido.php';
require_once __DIR__ . '/../src/Pedido.php';
```

O método `paraArray()` existe em todas as classes que aparecem no JSON (Produto e filhas, ItemPedido e Pedido) e é o que alimenta a rota `?formato=json`.

5 Camada web

As páginas são escolhidas por `?pagina=...`. Um `switch` no topo do `index.php` decide qual conteúdo será renderizado; se o valor não existir, a resposta recebe `http_response_code(404)`. O mesmo cabeçalho e menu aparecem em todas as páginas por meio de `include`.

A rota `?formato=json` monta um array com todos os produtos e pedidos, chama `json_encode` e define `Content-Type: application/json; charset=utf-8`.

O formulário da página `?pagina=pedido` usa `POST` para criar um `ItemPedido`. Se alguma regra recusar os dados, a mensagem da `Exception` é exibida na página (dentro de um `catch`). Todo dado exibido passa por `htmlspecialchars` para evitar injeção de HTML.

Saída real do `curl.exe -i` na rota JSON:

```
HTTP/1.1 200 OK
Host: localhost:8000
Connection: close
X-Powered-By: PHP/8.5.12
Content-Type: application/json; charset=utf-8

{
    "produtos": [
        {
            "tipo": "ProdutoUnitario",
            "nome": "Queijo coalho",
            "preco": 35
        }
    ],
    "pedidos": []
}
```

Saída real do `curl.exe -i` em uma página 404:

```
HTTP/1.1 404 Not Found
Host: localhost:8000
Connection: close
X-Powered-By: PHP/8.5.12
Content-Type: text/html; charset=UTF-8

<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Feira Digital do Sertão</title>
    ...
    <h2>404 - Página não encontrada</h2>
...
```

6 Testes

O arquivo `teste.php` roda no terminal e mostra cada regra aceita e cada regra recusada. A saída abaixo comprova o funcionamento das regras R1–R5.

```
=== TESTE: Feira Digital do Sertão ===

--- 1. Polimorfismo ---
Queijo coalho          x2 => R$    70.00
Carne de sol           x2 => R$    85.80
Manteiga da terra      x2 => R$    44.80

--- 2. Regras de negócio ---
OK, recusado: Nome do produto não pode ser vazio.
OK, recusado: Preço deve ser maior que zero.
OK, recusado: Quantidade deve ser maior que zero.
OK, recusado: Produto perecível vencido não pode ser vendido.
OK, recusado: Pedido não pode ser finalizado vazio.
OK, recusado: Peso máximo deve estar entre 0 e 50 kg.

--- 3. Composição e cálculo ---
Itens: 2
Total: R$ 134,35
```

Cada linha "OK, recusado: ..." comprova que a exceção foi lançada no ponto correto e capturada pelo `catch`. Cada linha de cálculo comprova o polimorfismo: o mesmo método `calcularPreco()` devolve valores diferentes para produtos de tipos diferentes.

7 Ferramentas utilizadas

Ferramenta	**Para que foi usada**
PHP 8.5	Execução do servidor embutido e do `teste.php`.
VS Code	Edição do código e terminal integrado.
Git e GitHub	Versionamento, issues, branches, PRs e tags.
curl	Teste das rotas HTTP (`-i` para ver cabeçalhos).
Navegador e DevTools	Visualização das páginas e inspeção das respostas.
LLM (ChatGPT / Claude)	Apoio ao aprendizado de persistência em JSON e revisão de código.
OBS Studio	Gravação do vídeo explicativo.

8 Uso de LLM

O diário completo está em `docs/llm.md`, com cinco interações registradas contendo prompt, resumo da resposta e o que foi aceito, modificado ou rejeitado. Em resumo:

- Usei a LLM para **entender** como persistir dados em JSON com `file_put_contents` e `json_decode` — esse conteúdo não foi visto em aula e é a parte bônus do trabalho.

- Usei para **revisar** o método `calcularPreco()` das filhas de `Produto`, conferindo se a chamada a `parent::calcularPreco()` fazia sentido.
- Usei para **entender mensagens de erro** do PHP (`strict_types` reclamando de tipos `float` e `int`).
- **Erro da LLM que corrigi**: em uma das respostas, a ferramenta sugeriu colocar `public array $itens = [];` em `Pedido`, o que viola o encapsulamento (OO4). Troquei por `private array $itens = [];` e criei `getItens()` para leitura.
- **Sugestão rejeitada**: a LLM propôs usar `match` em vez de `switch` na rota de páginas; mantive `switch` porque é o que foi visto em aula e fica mais legível para o professor avaliar.

Reflexão: a LLM foi útil para destravar tópicos que não vi em aula (principalmente persistência em JSON) e para revisar código, mas ela também sugeriu soluções que violavam os requisitos do trabalho — por isso, toda sugestão foi lida com o enunciado ao lado. Aprendi que a LLM é boa para *explicar* e *questionar*, mas a responsabilidade pela linha final é sempre minha.

9 Dificuldades, aprendizados e conclusão

A maior dificuldade foi implementar a persistência em JSON sem ter visto o conteúdo em aula: precisei entender a diferença entre `json_encode` e `json_decode`, o uso de `JSON_PRETTY_PRINT` e o fato de que a leitura do arquivo pode retornar `null`, o que quebra o array. Resolvi tratando o retorno com o operador de coalescência nula.

Outra dificuldade foi decidir onde colocar cada regra de negócio: se no construtor, se no setter, ou se no método que usa o objeto. Optei por colocar as regras mais próximas do ponto onde o dado é usado — quantidade em `ItemPedido::__construct()`, validade em `Pedido::adicionarItem()` — porque assim a mensagem de erro fica mais clara.

Aprendi na prática o que significa **polimorfismo**: o mesmo `foreach` chama `calcularPreco()` em objetos de classes diferentes e cada um responde do seu jeito, sem que o código chamador precise saber o tipo concreto. Aprendi também a importância do `declare(strict_types=1)` e como o PHP reclama quando passamos `int` onde se espera `float`.

Se fizesse diferente, teria começado a modelagem pelo diagrama Mermaid antes de escrever qualquer linha de código, porque o diagrama ajuda a enxergar a hierarquia e a composição com clareza. Também teria aberto as issues do GitHub mais cedo, uma por funcionalidade, para que os commits ficassem distribuídos ao longo do prazo, e não concentrados perto da entrega.

Concluo que o sistema atende a todos os requisitos OO1–OO8 e W1–W5, com bônus de persistência em JSON implementado. O código está organizado em uma classe por

arquivo, a hierarquia é clara, a composição entre `Pedido` e `ItemPedido` funciona, e as regras de negócio recusam dados inválidos com mensagens exibidas ao usuário.

Links

- Repositório: https://github.com/fernandalaralves/Feira_Digital_do_Sertao